

SYSTEM DESIGN

AES-Based Secure File Encryption and Decryption Tool

Maralbyek Tilyek

Table of Content

1. Introduction.....	2
2. System Overview.....	2
3. System Requirements.....	2
3.1 Functional Requirements.....	2
3.2 Non-Functional Requirements.....	3
4. System Architecture.....	4
5. System Design Diagrams.....	4
5.1 Use Case Diagram.....	5
5.2 Data Flow Diagram.....	5
5.3 Application Flow Diagram.....	7
6. Encryption Process Design.....	8
Step 1 – File and Password Input.....	8
Step 2 – Salt Generation.....	8
Step 3 – Key Derivation.....	8
Step 4 – Nonce Generation.....	8
Step 5 – AES-GCM Encryption.....	8
Step 6 – File Formatting.....	8
Step 7 – Completion.....	8
7. Decryption Process Design.....	8
Step 1 – Encrypted File Selection.....	8
Step 2 – File Parsing.....	8
Step 3 – Key Derivation.....	9
Step 4 – Authentication and Decryption.....	9
Step 5 – Success or Failure.....	9
8. Cryptographic Design.....	9
8.1 Password and Key Derivation.....	9
8.2 AES-256-GCM.....	9
8.3 Salt and Nonce Generation.....	9
8.4 Authentication Tag.....	9
9. Encrypted File Format.....	10
10. User Interface Design.....	10
11. Error Handling and Security Controls.....	11
12. Testing and Evaluation Design.....	11
13. Design Limitations.....	12
14. Conclusion.....	12
15. References.....	13

1. Introduction

This system design report will include the detailed design of the proposed AES Based Secure File Encryption and Decryption Tool. The system is a local desktop application designed to ensure the protection of files with password-based encryption keys and authenticated encryption. The design is based on the project proposal and literature review and translates the selected cryptographic concepts into an implementable application structure.

The proposed system employs Python for the local backend server, an HTML/CSS/JavaScript browser frontend for the user interface, PBKDF2-HMAC-SHA256 as a password-based key derivation function and AES-256-GCM as an authenticated encryption module. It is designed to be used by one user from one computer and doesn't require cloud storage, network transmission or a separate key-management server.

2. System Overview

The application will accept an encrypted .enc file or a normal file to be encrypted. On encryption, user chooses a file and enters a password. The system will create a random salt and nonce, calculate the 256-bit AES key from the password, encrypt the file with AES-256-GCM, and write the necessary non-secret parameters along with the encrypted file to a specifically-defined binary file format.

The application reads the encrypted file, retrieves the stored salt, nonce, authentication tag and ciphertext, generates same AES key from the provided password and performs AES-GCM authentication and decryption during decryption. On successful authentication, the original file is reconstructed. In case of failure of authentication, the application will not release or save the decrypted output.

3. System Requirements

3.1 Functional Requirements

ID	Requirement	Description
FR1	File Selection	The system shall allow the user to select a file through a file-picker dialog.
FR2	Password Input	The system shall accept a password and mask the password characters in the interface.
FR3	Password Confirmation	The encryption interface shall request password confirmation to reduce typing errors.

FR4	Encryption	The system shall derive an AES-256 key and encrypt the selected file using AES-GCM.
FR5	Encrypted File Creation	The system shall create a separate encrypted output file with a .enc extension.
FR6	Decryption	The system shall decrypt an encrypted file after successful authentication.
FR7	Integrity Verification	The system shall reject an incorrect password or modified encrypted data.
FR8	Output Protection	The system shall avoid creating final decrypted output until authentication succeeds.
FR9	Status Messages	The system shall display clear success and error messages.
FR10	Error Handling	The system shall handle missing files, invalid formats, permission errors, and interrupted operations without crashing.

3.2 Non-Functional Requirements

Category	Requirement
Security	Use AES-256-GCM, random salts, fresh nonces, and PBKDF2-HMAC-SHA256.
Usability	Provide a simple graphical interface understandable by a non-technical user.
Reliability	Successful encryption and decryption shall preserve file contents exactly.
Performance	The implementation shall be evaluated across multiple file sizes, including large files.
Maintainability	Cryptographic operations shall be separated from the GUI where practical.
Portability	The application shall run on supported Python 3.10+ environments with required dependencies.

4. System Architecture

The application's design is a layered logical design. The presentation layer is the browser-based frontend, where the user selects files and enters passwords. The application layer is responsible for controlling the encryption and decryption process, checking the input, handling the file operation, and reporting the results. Cryptographic layer uses Python cryptography library for PBKDF2 key derivation and AES-256-GCM operations. The file layer is responsible for reading and writing the file format used for the encrypted data.

Layer	Main Responsibility
Presentation Layer	Presentation Layer HTML/CSS/JavaScript browser interface, file selection, password entry, progress/status messages.
Application Layer	Workflow control, validation, temporary-file handling, success/failure decisions.
Cryptographic and File Layer	PBKDF2, AES-GCM, secure random generation, encrypted file formatting and parsing.

5. System Design Diagrams

The system is described from three different viewpoints through three diagrams. The Use Case Diagram illustrates the capabilities of the user of the system. A Data Flow Diagram depicts data flows between processes and data stores. The Application Flow Diagram illustrates the order of operations and decisions made while encrypting and decrypting.

5.1 Use Case Diagram

From the Use Case Diagram, the user is identified as a primary external actor. The primary purpose of the use-cases is to select a file, enter a password, encrypt a file, decrypt a file, select a destination, and get feedback on success or failure.

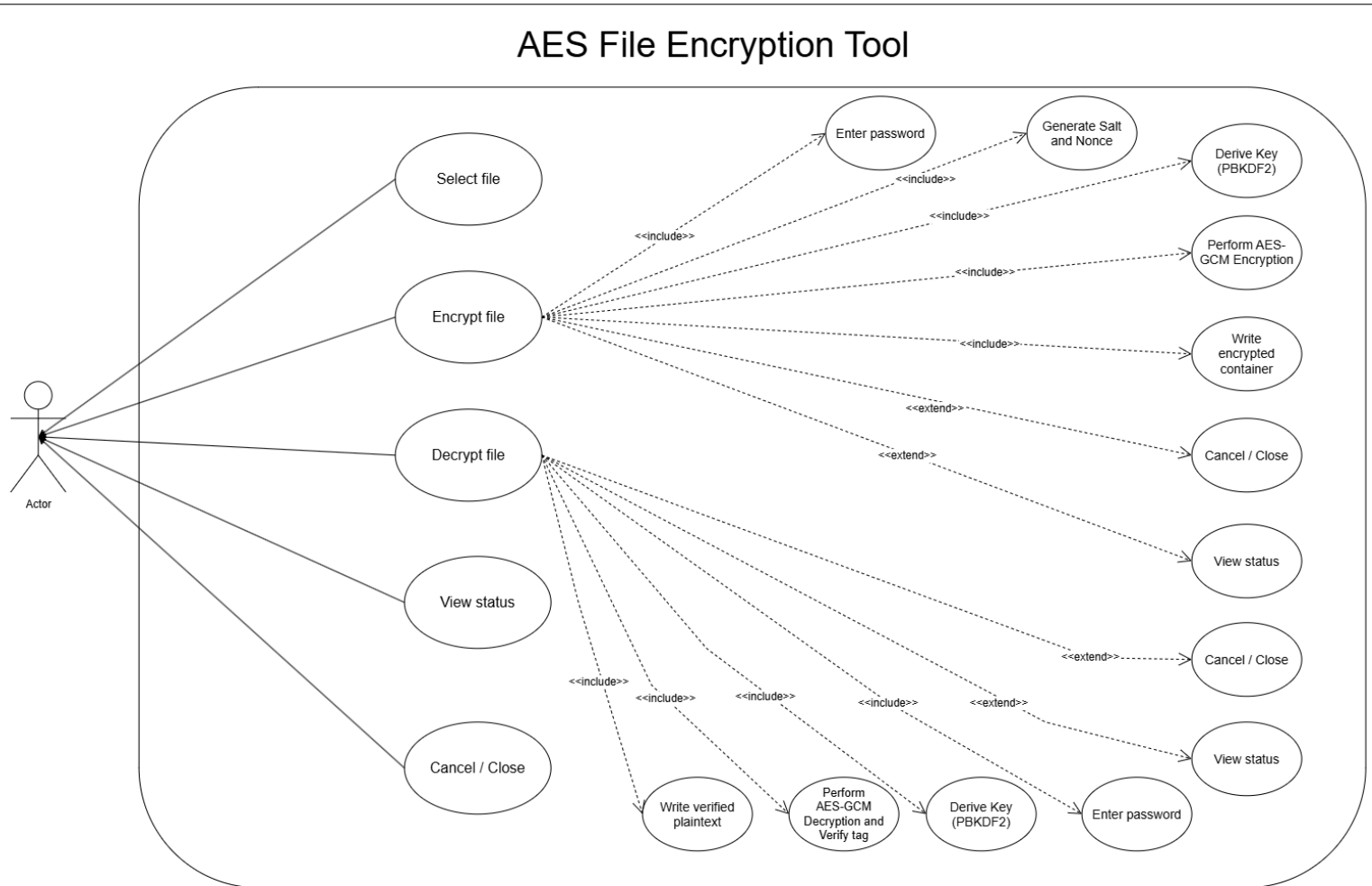


Figure 1. Use Case Diagram of the AES-Based Secure File Encryption and Decryption Tool.

5.2 Data Flow Diagram

The Data Flow Diagram represents the movement of files, passwords, derived keys, and encrypted data through the system. The main processes are file selection and password input, key derivation, AES-GCM encryption, encrypted file formatting, encrypted file parsing, key derivation during decrypt.

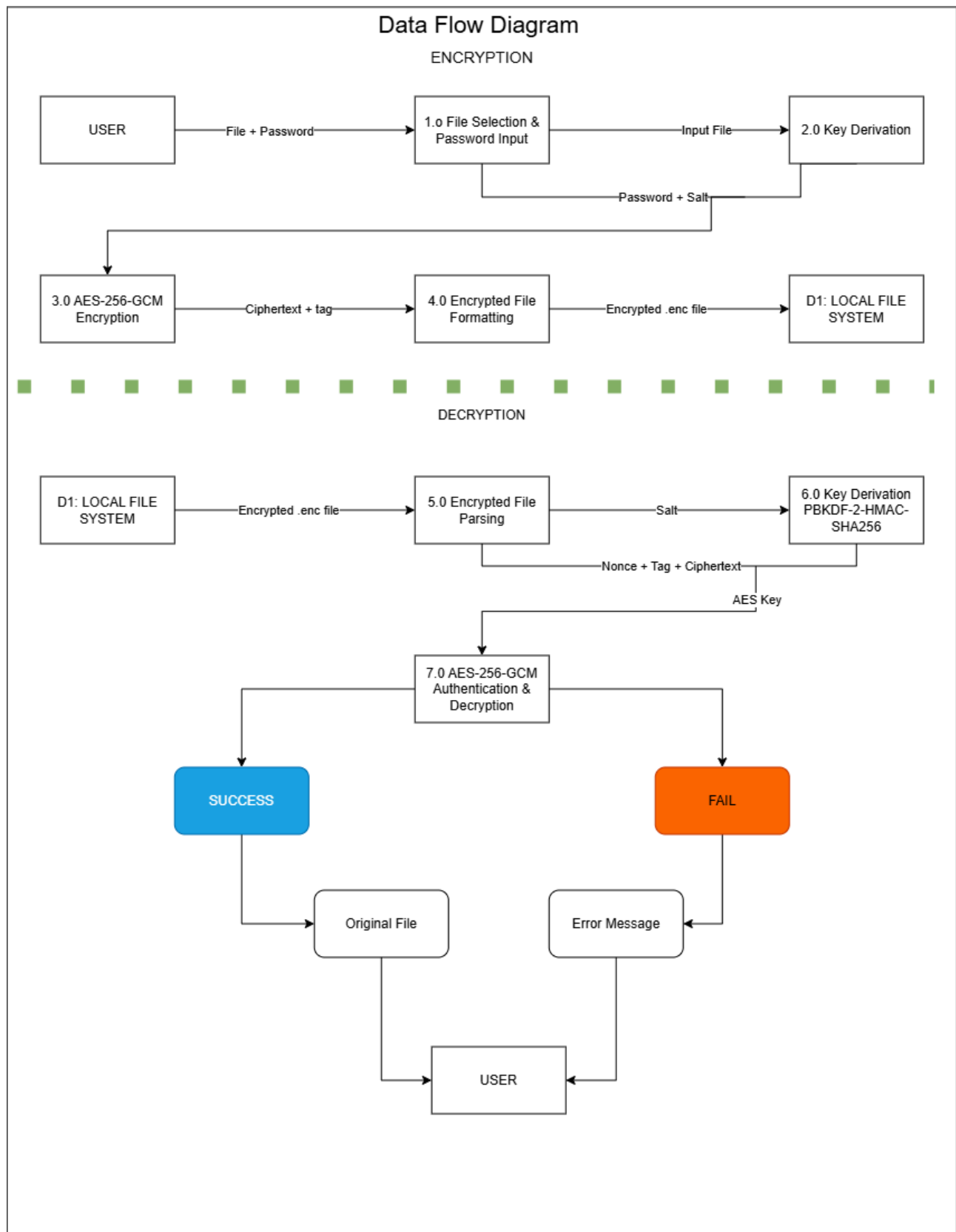


Figure 2. Data Flow Diagram of the AES-Based Secure File Encryption and Decryption Tool.

5.3 Application Flow Diagram

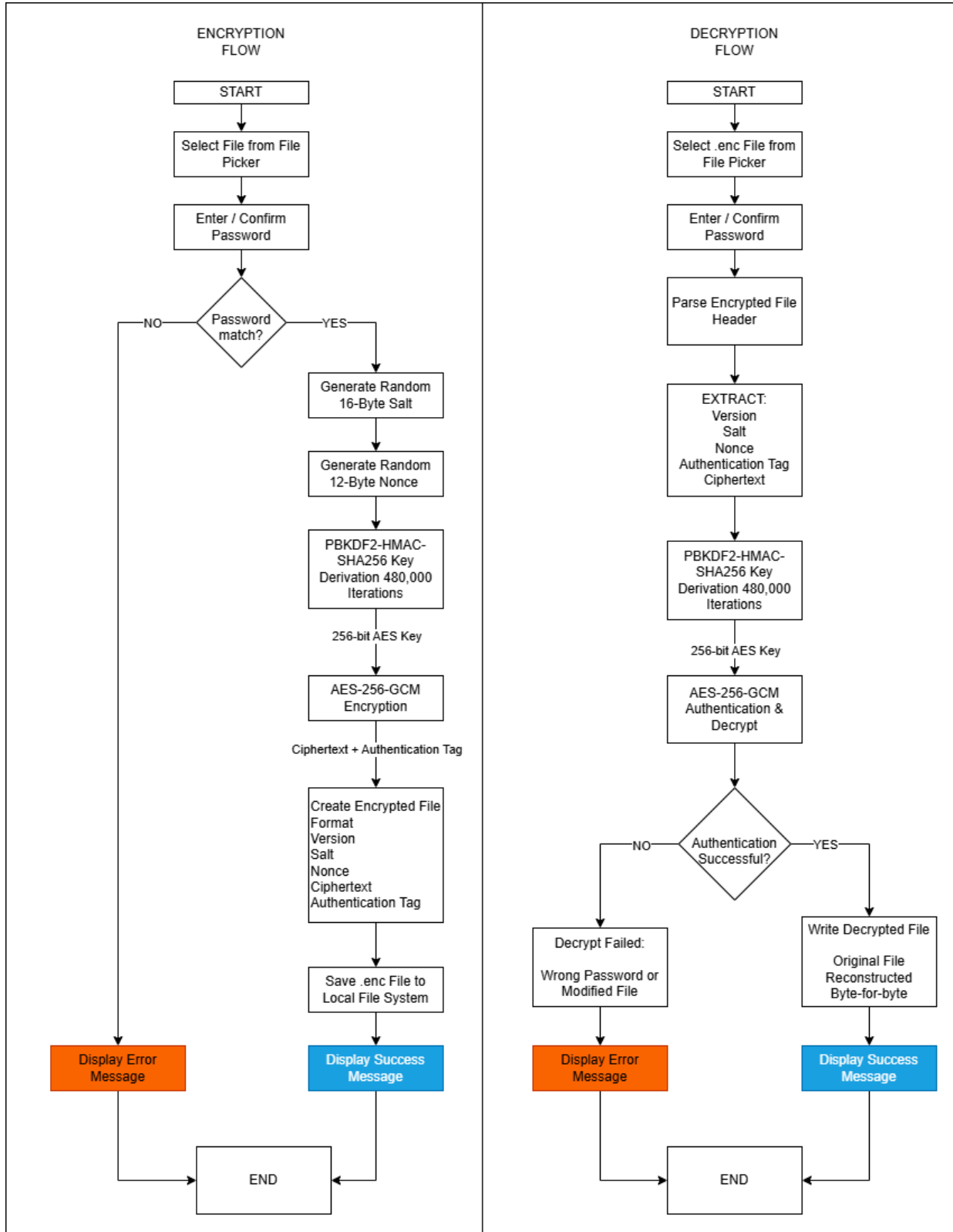


Figure 3. Application Flow Diagram for Encryption and Decryption.

6. Encryption Process Design

Step 1 – File and Password Input

User enters a password twice and chooses a source file. Application verifies file and both passwords.

Step 2 – Salt Generation

A cryptographically secure random 16 byte salt is generated. The salt is stored with the encrypted file as it is needed for future key derivation.

Step 3 – Key Derivation

The salt is combined with the password in this same manner, and then the resulting hash is passed into PBKDF2-HMAC-SHA256 with the number of iterations, generating a 32-byte (256-bit) AES key.

Step 4 – Nonce Generation

A new 12-byte nonce is used for AES-GCM. The nonce is attached to the encrypted data, but is not considered to be secret.

Step 5 – AES-GCM Encryption

The AES-256-GCM is used to encrypt the bytes of the plaintext file. The result is a 16-byte authentication tag and ciphertext.

Step 6 – File Formatting

The application appends the version/metadata fields, salt, nonce, authentication tag and ciphertext to the encrypted output file.

Step 7 – Completion

The original file is not altered, and the encrypted one is stored in a separate file.

7. Decryption Process Design

Step 1 – Encrypted File Selection

Step 1 is the selection of an encrypted file. Step 1 is the choice of an encrypted file.

User chooses a file in the appropriate expected encrypted format and provides the password.

Step 2 – File Parsing

The application verifies the file header, and decrypts the salt, nonce, authentication tag, and ciphertext.

Step 3 – Key Derivation

The salt and the password are passed through PBKDF2-HMAC-SHA256 with the salt and derivation parameters stored.

The next step is authentication and decryption. The next step is authentication and decryption.

Step 4 – Authentication and Decryption

AES-GCM decrypts the ciphertext by checking the authentication tag. Only if the authentication succeeds, the result is accepted.

Step 5 – Success or Failure

If the authentication is successful, the plaintext is saved to the selected destination. No final decrypted file will be created, and an error will be displayed if authentication fails.

8. Cryptographic Design

8.1 Password and Key Derivation

The user's password is NOT used directly as the AES key. The password is used to generate 32 bytes of key using PBKDF2-HMAC-SHA256. The password is salted and hashed to a 32-byte key using PBKDF2-HMAC-SHA256. Different salts are used for each encryption. The design follows the number of iterations mentioned in the Project Proposal and the implementation stores the iteration count in the encrypted file metadata to allow the same derivation process to be replicated.

The password must never be saved in the encrypted file. The non-secret salt and parameters needed to reproduce this derivation are stored.

8.2 AES-256-GCM

AES-256-GCM is a combination of confidentiality and authentication. AES has a 256-bit key, and GCM generates an authentication tag that can be used by the application to recognize changes to encrypted data. Instead of implementing AES, the Python cryptography library will be used.

8.3 Salt and Nonce Generation

A cryptographically secure random number source is used to generate the salt and nonce. The proposed design is based on a 16-byte salt and a 12-byte nonce. The salt can only be used as metadata for the salted file to be encrypted, and a different salt is created for each subsequent encryption operation. A new nonce is created for each AES-GCM encryption process.

8.4 Authentication Tag

The authentication tag is a value of critical importance for security purposes. It is kept along with the encrypted data and it has to be provided without any change in the decryption process. When the tag

verification is not successful, the operation is assumed as failed. This includes wrong password and changing the plaintext or any of the other authenticated encryption inputs.

9. Encrypted File Format

The encrypted file is in a self-contained binary format, meaning that there is no key file needed. To determine changes to the format, a version field is recommended for future application versions.

Field	Example Size	Purpose	Secret?
Magic / Version	Variable	Identifies the file type and format version.	No
PBKDF2 Parameters	Variable	Stores parameters required to reproduce key derivation.	No
Salt	16 bytes	Used with the password to derive the AES key.	No
Nonce	12 bytes	Required for the AES-GCM operation.	No
Authentication Tag	16 bytes	Used to verify ciphertext authenticity.	No
Ciphertext	Variable	Encrypted contents of the original file.	Protected

It is important to record the exact layout of the bytes, and the encoding of their length, in the implementation, so that parsing is deterministic and malformed files can be rejected safely.

Evaluating a design through testing and refinement. Testing and development of design.

10. User Interface Design

Interface Component	Design
Main Window	Provides clear choices for Encrypt and Decrypt operations.
File Selection	Uses a file-picker dialog and displays the selected filename/path.
Password Entry	Uses masked input. Encryption includes password confirmation.
Progress/Status Area	Shows the current operation without exposing cryptographic secrets.
Result Messages	Reports successful completion or a general failure reason.
Output Selection	Allows the user to choose where the resulting encrypted or decrypted file is saved.

11. Error Handling and Security Controls

Condition / Risk	Control
Wrong Password	AES-GCM authentication failure is returned as a general decryption failure. No plaintext output is created.
Modified Ciphertext	Authentication fails and the encrypted file is rejected.
Malformed File	Header and length validation are performed before cryptographic processing.
Missing Source File	The application reports that the selected file cannot be accessed.
Permission Error	The application displays a user-friendly message instead of terminating unexpectedly.
Interrupted Write	Temporary output is used and the final file is created/renamed only after successful completion.
Password Exposure	Password values are masked and are not stored in the encrypted file or ordinary application logs.
Accidental Overwrite	The application should ask for confirmation before overwriting an existing destination.

12. Testing and Evaluation Design

The correctness, security, usability, performance and robustness will be assessed by testing. Test will be to compare the original file and the recovered one using the hash function SHA-256 and will prove that the recovered file is the same as the original one bit by bit. Insecurity scenarios will include intentionally using wrong passwords and altering encrypted data to ensure that authentication failures are dealt with appropriately.

Test	Method	Expected Result
Round-trip test	Encrypt then decrypt a file and compare SHA-256 hashes.	Original and decrypted hashes match.
Wrong password	Attempt decryption with an incorrect password.	Authentication fails and no final plaintext file is produced.
Tampered ciphertext	Modify one or more ciphertext bytes.	Authentication fails.
Tampered tag	Modify the authentication tag.	Authentication fails.
Different file types	Test TXT, PDF, image, and archive files.	All files decrypt correctly.
File sizes	Test small and large files, including performance benchmarks.	Operations complete without corruption.

Missing file	Remove or move the selected input before operation.	Clear error and no crash.
Interrupted operation	Simulate an interrupted write where practical.	No misleading incomplete final output.

13. Design Limitations

The system is not an enterprise key-management system, but an academic desktop prototype. It does not support password recovery, cloud integration, multiple user access control, hardware security modules or against attackers who already possess an unlocked running system. Passphrase-based encryption is also highly dependent upon the strength of the user's passphrase. These constraints are deliberately placed and are consistent with those set out in the project proposal.

14. Conclusion

The system design translates the requirements for cryptography of the suggested project into a usable desktop design. It is a simple browser-based frontend served locally by a Python HTTP server that uses PBKDF2-HMAC-SHA256 key derivation and AES-256-GCM authenticated encryption. For each encryption operation, random salts and fresh nonces are generated; the non-secret information for future decryption is contained within the encrypted file. The output is considered to be authenticated before it is issued.

The three system diagrams (Use Case Diagram, Data Flow Diagram, and Application Flow Diagram) are complementary views of the system, and will be included in this report prior to submission. They come with the requirements, cryptographic design, file format, interface design, error controls, and test plan and form the basis for the implementation phase.

15. References

- Dworkin, M. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-38D>
- Kaliski, B. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). Internet Engineering Task Force. <https://doi.org/10.17487/RFC8018>
- National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)* (FIPS PUB 197). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.197>
- National Institute of Standards and Technology. (2010). *Recommendation for password-based key derivation: Part 1, storage applications* (NIST Special Publication 800-132). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-132>
- OWASP Foundation. (n.d.). *Cryptographic storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- OWASP Foundation. (n.d.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Python Cryptographic Authority. (n.d.). *Authenticated encryption with associated data (AEAD)*. cryptography. <https://cryptography.io/en/latest/hazmat/primitives/aead/>